ML Based Noise Reduction

Earl Wong

Overview

- In VISION_NoiseReduction, we walked through different classical methods for performing noise reduction.

- In practice, BM3D provides a state of the art anchor for classical noise reduction methods.

- Hence, BM3D is frequently used as a reference for emerging ML based techniques.

Overview

- State of the art ML based noise reduction models leverage contributions from noise modeling, model architecture, and model training.
- A clear division exists between ML based spatial and ML based temporal noise reduction.
- Generative diffusion provides a nice “link” to ML based noise reduction.
- However, generative diffusion is a one to many process, resulting in a distribution of clean output images for a noisy input image.
- i.e. ML denoising is a deterministic process while generative diffusion is a stochastic process.

ML Based Models

Three main categories

- Noise modeling
- Model architecture
- Model training

Noise Modeling

- ML denoising models fall into two categories - blind and non blind.
- In blind denoising, the ML model is essentially a black box that has learned the appropriate denoising for EVERY conceivable input image.
- From a “prior example” argument, one can argue that this may not be an unreasonable expectation.
- Prior example: Given the success of transformers (i.e. their ability to learn all of the inductive biases inherent in CNN’s for tasks like image classification), the necessary conditions appear to be 1) enough training data and 2) a model with adequate capacity.
- i.e. Everything is learned / scale trumps all = ViT classifiers outperform their CNN counterparts.

Noise Modeling

- However, for image denoising, the underlying task complexity is significantly higher.

Pixel Level Ambiguity (Gaussian Noise)

- Suppose we observed a pixel value of 170.
- Is this value associated with a noise free image? Or, was the original pixel value 150 with +20 being additive noise? Or, was it some other combination?

With Potential Signal Dependence (Shot Noise)

- In practice, at 8 bit resolution for an NxN image, there are N*N*256 possible pixel combinations.
- When non-stationary noise is added / introduced, the complexity of the estimation problem explodes, even though the cardinality of the output remains the same.

Noise Modeling

With Correlations and Skew

- Shot and gaussian noise begin as independent entities at the pixel level.

- However various ISP operations (de-mosaicing and tone mapping) then make the noise correlated and skewed.

Noise Modeling

- To make a complex problem more tractable, solvable, and meaningful, constraints are added.

- Let’s start with meaningful, and the pixels in the input image themselves.

- If the image was purely random noise, the image would contain no useful information.

- Codecs would then be ineffective / unnecessary, because there would be nothing to compress.

- Fortunately, (useful) images contain local correlation between neighboring pixels.

Noise Modeling

- With local correlation, the space of N*N*256 images reduces to the space of natural images.
- Hence, the cardinality is now magnitudes of order smaller.
- How do we make the noise removal problem more solvable?
- -By constraining the input noise space.

Noise Modeling

- Zhang, et. al. provided a means of accomplishing this in Fast and Flexible Denoise Net (FFDNet), with the introduction of a noise map.

- Here, information about the type of noise, the noise level, and its non-stationarity could be infused into the learned model.

- This provided a significant advance, to the goal of shipping an ML based denoising model.

- In contrast, although blind de-noising is a very interesting problem, its lack of constraints / “openness” make “shipping a product” challenging.

Noise Modeling

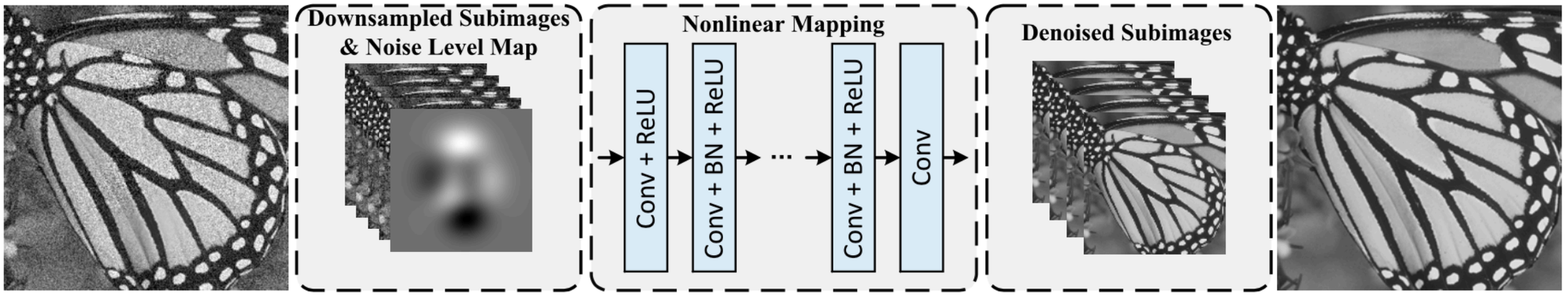


Fig. 1. The architecture of the proposed FFDNet for image denoising. The input image is reshaped to four sub-images, which are then input to the CNN together with a noise level map. The final output is reconstructed by the four denoised sub-images.

The noise map injects valuable noise related information into the training process.

Hence, the ML model is no longer burdened with “figuring it out”.

Noise Modeling

The Noise Model Details - A to Z

- Consider an image sensor and a single pixel location.
- Photons are collected at this pixel location.

What affects the number of photons collected?

- The radiance, the aperture, the micro lens, the pixel size, and the exposure time.

Noise Modeling

What affects the number of photoelectrons generated?

- The quantum efficiency, η , which is a function of wavelength.

- $\lambda_e = \eta \lambda_{photon}$

- By construction, the mean pixel value and the resulting shot noise, will be different for R, G, and B pixels.

- Since the B pixels have the lowest quantum efficiency, the B pixels will have the lowest mean value, for a given radiance.

Noise Modeling

- Mathematically: $y(x_i, y_j) = x(x_i, y_j) + n_{shot}(x_i, y_j) + n_{read}(x_i, y_j)$

  where $x(x_i, y_j)$ *represents the mean pixel value .*

- For a poisson distribution, the mean = variance = λ.

- λ_e denotes the expected number of photoelectron pairs generated for a specific time interval.

- Hence: $y(x_i, y_j) = \lambda_e(x_i, y_j) + n_{shot}(x_i, y_j) + n_{read}(x_i, y_j)$

  n_{shot} *is a zero mean noise process with variance* λ_e

  n_{read} *is gaussian noise and is independent of scene brightness .*

Noise Modeling

- In practice, the brighter the scene radiance, the larger the λ_e and the greater the shot noise.

Notes

- $SNR = \frac{mean\ signal}{std\ noise\ variance} = \frac{\lambda_e}{\sqrt{\lambda_e}}$
- What this says: Although shot noise increases with scene brightness, the mean signal increases at an even faster rate.

- Hence, shot noise is only problematic in the low SNR scenario = low light scenario.

- For very low light (typically < 1 lux), read noise can become the dominant noise term, since the number of photoelectrons associated with read noise will (then) exceed that of shot noise.

Noise Modeling

- For the simplest use case - raw data capture immediately from the sensor with proper exposure time / no analog gain applied, we obtain a non-stationary sigma / noise map.

- Mathematically: $\lambda_e = \eta\lambda_{photon}$, *where* η *denotes quantum efficiency*

$$\sigma^2(x_i, y_j) = \lambda_e(x_i, y_j) + \eta^2_{read}(x_i, y_j)$$

Noise Modeling

- Adding analog gain g, we obtain: $\sigma^2(x_i, y_j) = g^2\lambda_e^2(x_i, y_j) + g^2\eta_{read}^2(x_i, y_j)$
  (This equation assumes that the gain occurs prior to the read noise.)

- At present, every pixel location is independent of every other pixel location.
  - After de-mosaicing, every pixel location becomes correlated with it's neighboring pixels.

Notes

- For a large enough λ_e (photoelectron count), the poisson distribution becomes a gaussian distribution, by the central limit theorem.

- When IID random variables are added, the distribution "evolves" to a gaussian distribution, also by the central limit theorem.

Noise Modeling

- Finally, consider the addition of tone mapping or gamma correction to each pixel location.

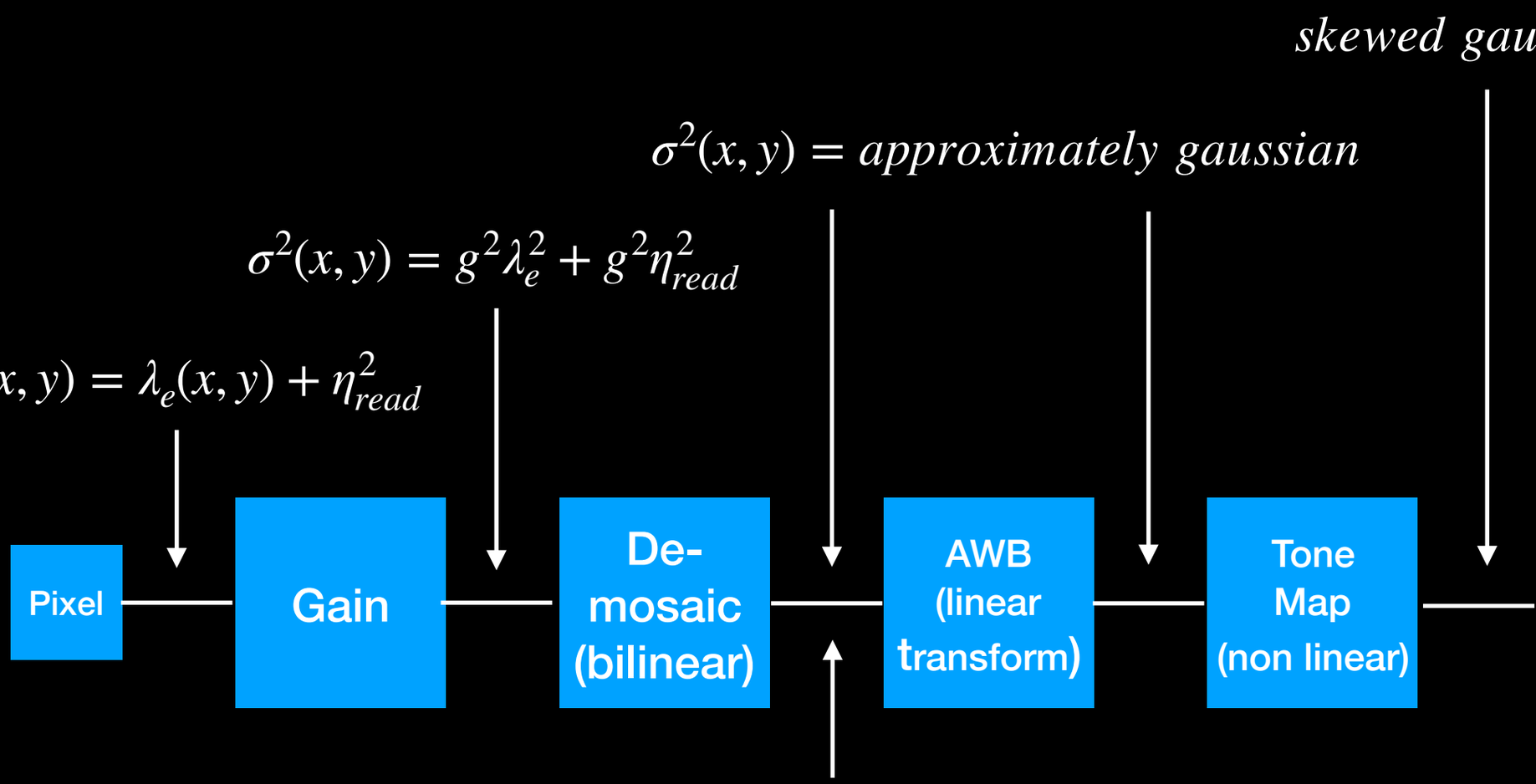
- Because of the non-linearity of the operation, a skewed gaussian distribution results.

- This is a lot for any ML model to ingest / figure out.

- Add ISP tuning, and you have a shipping nightmare.

- In practice, a network is trained for a given noise model, and the model is then applied at the location representative of the noise.

Noise Modeling



Pixel noise is no longer independent /
Pixel noise is spatially correlated;
The covariance matrix is no longer diagonal

If de-mosaicing is edge aware, etc., linearity is compromised.

Noise Modeling

- One of the most recent improvement to ML based denoising was presented by Oh, et. al., in 2025.
- Here, camera meta data parameters (exposure time, ISO, aperture) were fed into the ML model.
- This gave the ML model valuable information about the “noisiness” of the input image, further improving the efficacy of the trained model.

Noise Modeling

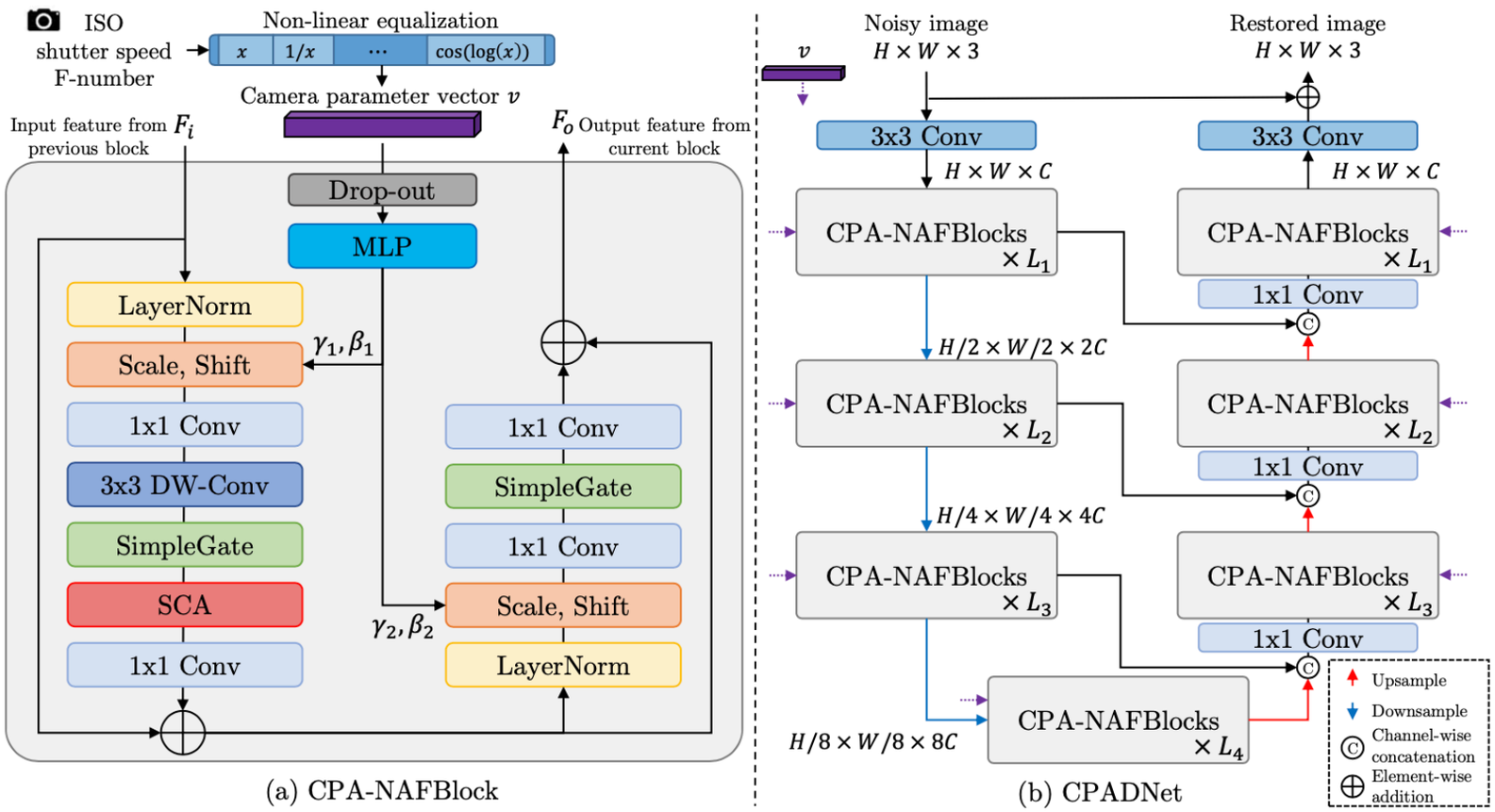


Fig. 2. (a) Proposed CPA-NAFBlock, which is modified from NAFBlock [7] with adaptive layer normalization [8], and (b) CPADNet, a denoising network composed of CPA-NAFBlocks. CPADNet is designed as a U-shaped network with skip-connections and a hierarchical encoder-decoder architecture [13] for its effectiveness in image restoration.

Model Architecture

- Next, let’s discuss the model architecture.
- We will focus on two different aspects.

1) The output of the CNN.

2) The CNN.

Model Architecture - DnCNN

- Originally, ML based denoisers were focused on generating noise free, output images.

- i.e. They paralleled the image restoration problem.

- This required the ML model to create a clean version of any / all image structure, using its learned representation.

- This proved to be a challenging task - for example: dark, low contrast areas of an image. (i.e. Hallucinations resulted / hallucinations occurred in the shadows.)

- To prevent hallucinations, the script was flipped from restoration to residual learning by Zhang, et. al., -> instead of generating a noise free output image, an output noise map was estimated.

Model Architecture - DnCNN

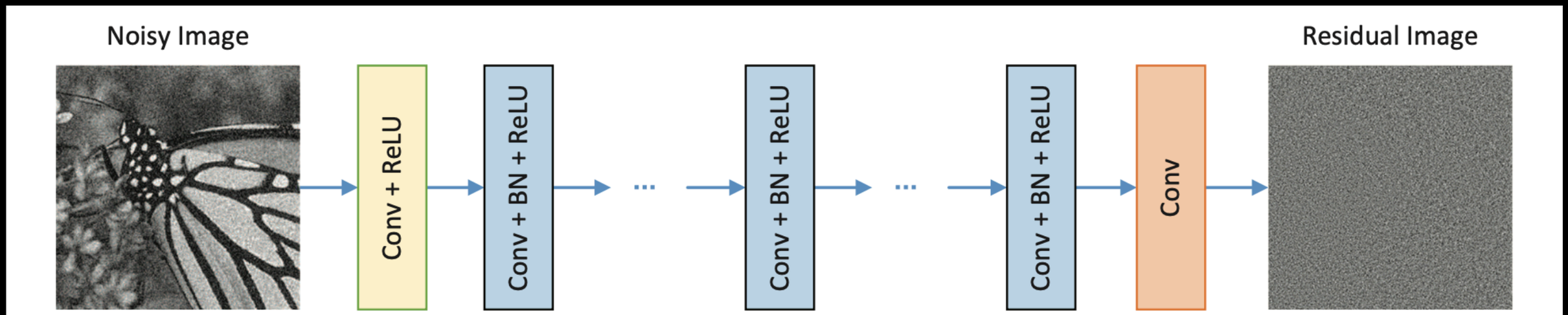


Fig. 1. The architecture of the proposed DnCNN network.

Denoising now occurred, by subtracting the estimated residual image from the original noisy image.

By construction, the new approach was not highly amenable to hallucinations, since the approach naturally "reduced" the energy in the resulting image.

Model Architecture

- When training an ML model for a specific task, tasks, or multiple modes, various architectures will result.
- For example, for (single task based) image classification, the CNN architecture evolved from AlexNet, to VGGNet, to Inception, to ResNet, to EfficientNet.
- In the previously mentioned FFDNet and DnCNN papers, a fully convolutional network was used.
- However, for many imaging tasks requiring full resolution pixel information, a more efficient UNet architecture exists.

Model Architecture - UNet

- The UNet was introduced by Ronnenberger, et. al. in 2015, as an alternative and improvement to the Fully Convolution Net by Shelhamer, et. al. in 2014 for image segmentation.

- Unlike the FCN, the UNet consisted of an encoder-decoder architecture, with a bottleneck layer.

- By construction, the architecture was more computationally efficient than the full convolutional layers found in FCN.

- However, the bottleneck affected the output quality.

- To address this problem, skip connections were added.

Model Architecture - UNet

- The concatenation of the encoder feature maps to the equivalent resolution decoder feature maps provided the “skip”.

- i.e. The “skip” is a concatenation skip and not an identity skip.

Advantages

- The concatenation skip preserved the quality of the details (spatial information) at every depth.

- i.e. The gradients no longer need to flow through the bottleneck (context / compression), improving training stability.

Model Architecture - UNet

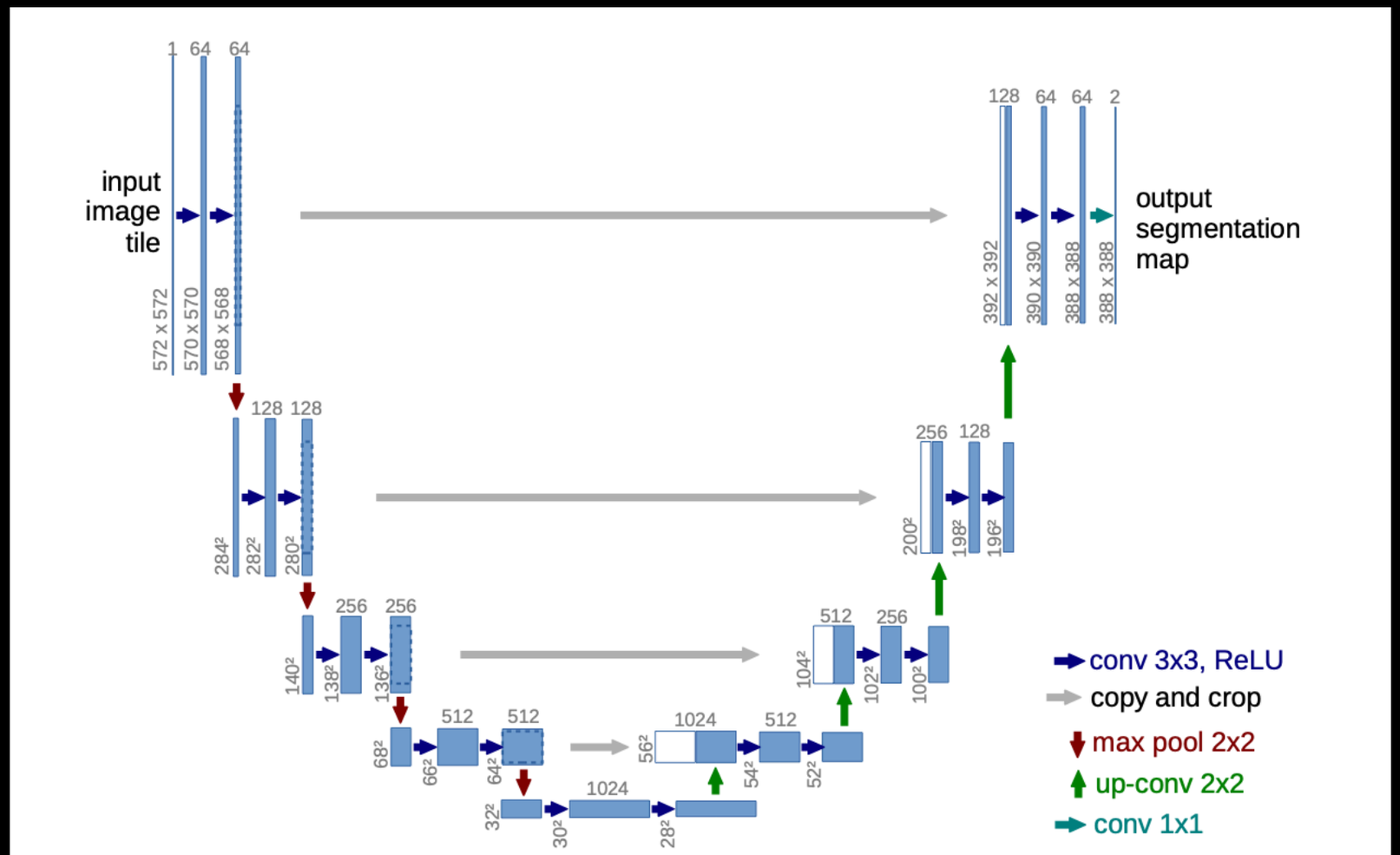


Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

Model Architecture

Regarding the learned representations

- Low level feature maps contain edges, gradients, and color information.
- They are naturally created at the highest input resolution, because of the nature of the convolution operator.
- Mid level feature maps contain textures, curves and object parts.
- They result from aggregating the information from the low level feature maps.
- High level feature maps contain semantic information / semantic abstractions.

Model Architecture

Regarding the learned representations

- FCN’s operate at a fixed resolution.
- FCN’s also contain low, mid, and high level information.
- However, this information is now encoded at every spatial location.
- i.e. The high level feature maps now encode semantic information at every spatial location.
- This makes FCN’s well suited for segmentation tasks, at the expense of computational cost.

Model Architecture

Regarding the learned representations

- For denoising, a UNet with skip connections significantly reduces the computational cost, while still maintaining the needed information necessary for effective denoising.

- For de-noising, the high level feature maps now contain context aware information.

- In addition, because of differing objectives, the mid and high level feature maps will now contain the learned representation needed for EITHER residual noise prediction OR noise free structure generation.

Model Training

- Model training can occur is several different ways.
- Supervised learning: image pair = (clean image, noisy image)
- Self supervised learning: image pair = (noisy1_imageA, noisy2_imageA)
- Self supervised learning / blind spot learning: single image

Model Training

Supervised

- General formulation: (input image, ground truth output)
- Ground truth output = label (classification)
- Ground truth output = noise free image (denoising)
- Cons: Ground truth output is expensive to obtain

Model Training

Self Supervised (image pair)

- Fascinating, from a mathematical standpoint.

- The same de-noising model is learned using an image pair consisting of: a noise free input image and two different levels of noise.

- When the noise level for one input image is 0, we converge to supervised learning.

Model Training

Self Supervised (image pair)

- Major assumption: Noise is zero mean

- Major assumption: Loss is MSE

- Important fact that “makes it work”: minimizing functionA versus minimizing functionA + constant, does not change the minimization problem / does not change the end result

Model Training

$y \sim$ *clean image* $\quad y_1 = y + \epsilon_1 \quad y_2 = y + \epsilon_2 \quad y_3 = y + \epsilon_3$

$E[\epsilon_1]$ *and* $E[\epsilon_2]$ *and* $E[\epsilon_3]$ *are zero mean*

$\rightarrow min_\theta E[L(f_\theta(y_1), y_2) \,|\, y_1] = min_\theta E[L(f_\theta(y_1), y_3) \,|\, y_1]$

Proof:

$min_\theta E[L(f_\theta(y_1), y_2) \,|\, y_1] \rightarrow E[\,||f_\theta(y_1) - (y + \epsilon_2)||^2]$

$= E[\,||f_\theta(y_1) - y||^2] + 2(f_\theta(y_1) - y)E[\epsilon_2] + E[\,||\epsilon_2||^2]$

$min_\theta E[L(f_\theta(y_1), y_3) \,|\, y_1] \rightarrow E[\,||f_\theta(y_1) - (y + \epsilon_3)||^2]$

$= E[\,||f_\theta(y_1) - y||^2] + 2(f_\theta(y_1) - y)E[\epsilon_3] + E[\,||\epsilon_3||^2]$

In both cases, the estimated noise model is not influenced by the constant term => the minima location does not change from a shift.

Model Training

Self Supervised (single image)

- The authors use an in painting approach to perform denoising.

- Although the approach underperforms the previous self supervised approach, the method is very interesting.

- Assumption: The signal is correlated / the neighboring pixels are correlated (otherwise, nothing useful could be learned).

- Assumption: The noise is independent at each pixel location.

Loss Function

- L2
- L1
- Anscombe Transform

Model Training - Loss Function

- L2 is MSE loss, while L1 is SAD loss.
- Both are highly relevant to any / all regression problems.
- Both are objective loss functions versus perceptual loss functions.
- For deterministic ML model based denoising, the L2 loss results in the mean, while the L1 loss results in median.
- Although the loss is computed per pixel (using a reference value at each pixel location), the receptive field of the CNN is used to used to estimate each per pixel value.

Model Training - Loss Function

- For shot noise, the noise variance is signal dependent.
- i.e. Brighter pixels have higher noise variance, while darker pixels have lower noise variance.
- To ensure that the L2 loss function does not overweight the loss from brighter pixels, the Anscombe transform can be applied to the image.
- The Anscombe transform is a variance stabilizing transform, converting signal dependent shot noise into signal independent gaussian noise.
- Note: If an input noise map is injected into the ML denoising model, the Anscombe transform is no longer needed.

Still Image vs Video->Still

- Previously, we discussed residual learning - estimating the noise from a noisy image.

- In practice, residual learning is the preferred approach for deployment, because of its robustness to hallucinations.

- In contrast, the reconstruction of noise free structure is highly amenable to hallucinations.

- This is because 1) the reconstruction process relies on image priors and 2) the training process has no mechanism enabling it to say “I have no confidence. I choose to punt and do nothing.”

Still Image vs Video->Video

- Before discussing ML based video denoising, it is important to first discuss classical video denoising.

- Classical video denoising is typically an extension of classical spatial denoising.

- i.e. Spatial non local means or BM3D, extended to neighboring temporal frames.

Still Image vs Video->Video

- While spatial based ML denoising can leverage image priors and noise models, it lacks the very important temporal component.

- In practice, the temporal component is a double edged sword.

- Plus: Temporal denoising is evidence based / sample based, and rooted in statistics.

- Minus: Temporal denoising can create new artifacts, if applied incorrectly.

Plus

- For independent noise with zero mean and finite variance, the noise variance decreases with the number of observations.
- i.e. $\frac{\sigma^2}{N}$
- To obtain the full power of temporal denoising, it is good practice to perform temporal denoising on the raw data captured by the sensor.
- Otherwise, the full power of temporal denoising can become compromised.

Plus - With Caveats

- $\frac{\sigma^2}{N}$
- This relationship becomes compromised / less effective, once the noise becomes correlated, or, is biased.
- By definition, the noise is correlated, if: $Cov(\epsilon_i, \epsilon_j) \neq 0,\ i \neq j$
- Mathematically, the compromise is a function of the correlation coefficient ρ.

Plus - With Caveats

$$Cov(\epsilon_i, \epsilon_j) = \rho\sigma^2,\ i \neq j,\ \rho \in [-1,1]$$

$$Var[\frac{1}{N}\sum_i \epsilon_i] = \frac{1}{N^2}\sum_{i,j} Cov(\epsilon_i, \epsilon_j)$$

$$= \frac{1}{N^2}[\sum_i Var(\epsilon_i) + \sum_{i \neq j} Cov(\epsilon_i, \epsilon_j)]$$

$$= \frac{1}{N^2}[N\sigma^2 + N(N-1)\rho\sigma^2] = \sigma^2\frac{1+\rho(N-1)}{N}$$

Plus - With Caveats

- For an image, the noise becomes spatially correlated, once de-mosaicing or any filtering operation is applied.

- For an image sequence, the noise can become temporally correlated.

- Example: Suppose each new de-noised pixel is the weighted average of the current frame and an “average” of the previous N-1 frames ->

$$\alpha * current + (1 - \alpha) * (average\ of\ past\ frames)$$

- The averaging operation correlates the information in the previous N-1 frames.

Plus - With Caveats

- Noise can also become biased.

- This results from non-linearities (tone mapping / gamma, clipping, saturation, etc.) or from the creation of deterministic structures (sharpening halos, de-mosaicing artifacts, etc.)

- In these cases, no amount of averaging can recover the original structure / true signal.

- Note: If the noise is NOT zero mean, the noise (itself) is responsible for the bias.

- Note: If the noise IS zero mean, the process is responsible for the bias. For example: $y = (x + \epsilon)^{\gamma}$

Minus

- In temporal denoising, proper pixel alignment is critical.

Example

- Suppose you had a checkerboard pattern image, where each pixel corresponded to a square on the checkerboard.
- A one pixel shift (in the horizontal or vertical direction), would blur the image detail in that direction, when two consecutive frames were averaged.

Example

- Suppose you had a moving object in your captured temporal sequence.
- Strict averaging would result in a ghost outline, of the moving object.

Minus

- In practice, knowledge of any and all motion, needs to be accounted for and addressed, to obtain the best result.
- For example, a motion map with associated confidence values, could be computed and applied.
- A motion map could be obtained via optical flow, block matching, etc.
- For areas with low confidence, the corresponding pixels in the neighboring frames could be de-weighted (relative to the reference frame) during temporal averaging.

Minus - The Curse of Temporal Artifacts

- The human visual system is primed at identifying temporal incoherencies and inconsistencies.

- Because of this, non-smooth frame to frame variations become immediately evident.

Example

- Suppose we applied spatial based ML denoising to every frame, and a small hallucination occurred every ten frames or so, in the upper left corner.

- From a single image standpoint, the hallucination could easily go un-noticed - unless the image was zoomed.

- However, when viewed temporally, the “region popping” in the upper left corner (every ten frames or so) would become quickly apparent.

Minus - The Curse of Temporal Artifacts

Example

- Suppose the motion map was periodically inaccurate.
- Suppose the errors were not appropriately de-weighted with lower confidence scores.
- Certain areas of a video sequence then might oscillate between being smooth and non smooth - i.e. a pumping effect, tracking the motion map inaccuracies one to one.

Minus - Monday Morning / Ship Tomorrow Solutions

- Tuning knob to decrease the confidence weighting of the motion map, reducing the contributions / effects of incorrect information.

Improve the motion map by:

- Using multiple sources - optical flow, block matching, etc. to improve the confidence score.

- Using forward-backward motion map verifications, to identify occlusions and dis-occlusions regions.

Minus - Monday Morning / Ship Tomorrow Solutions

- Tuning knob, to restrict large variations / changes in areas of a scene deemed to be “flat” / small gradient.

- Tuning knob to set the window size / hysteresis loop to prevent any changes from occurring at all, unless a threshold is exceeded.

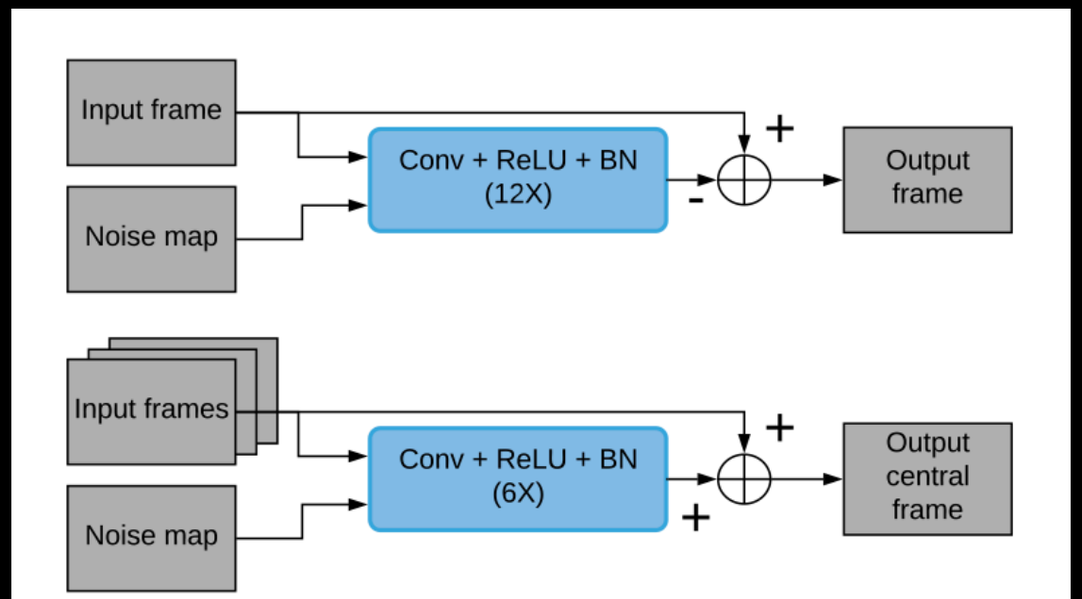
- => Once the threshold is exceeded, the change then becomes gradual, according to some smooth function vs an abrupt step function.

- Tuning knob to set a temporal smoothing filter width, to “los pass filter” the tracking of fast moving variations / fluctuations.

Still Image vs Video->Video

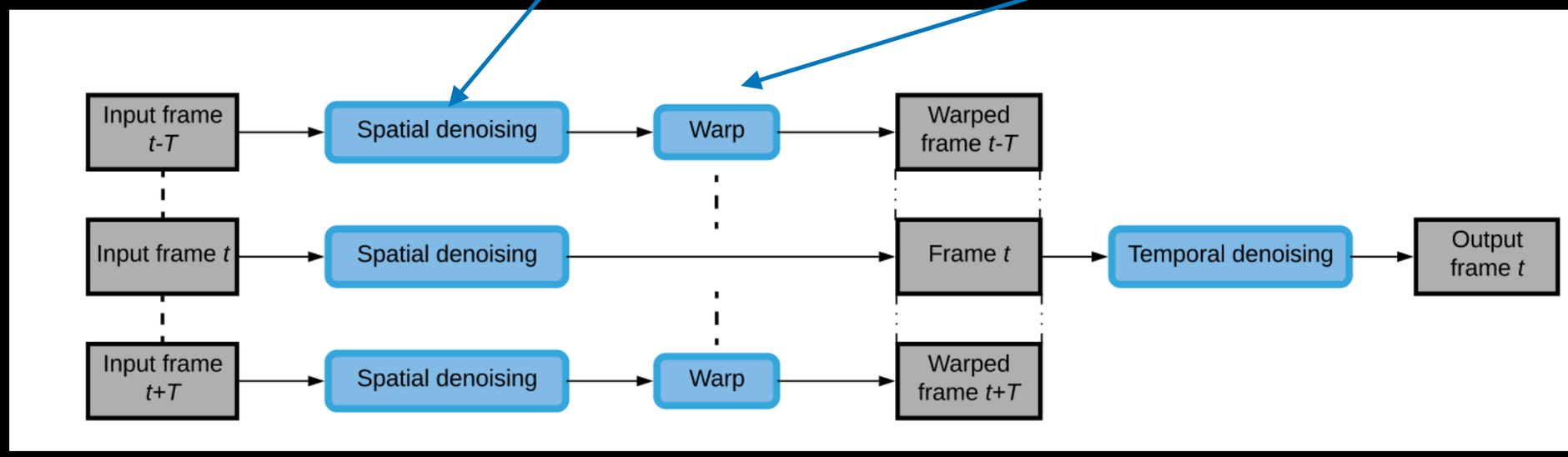
- DVDNet, built upon the motion map idea from classical video denoising, utilizing a two stage temporal denoising process.
- The first stage applied ML based spatial denoising to five consecutive input frames.
- Classical warping functions / motion maps were then applied to the neighboring -2, -1, +1, + 2 frames.
- All five spatially de-noised and aligned frames were then fed into a second stage ML denoiser.

Still Image vs Video->Video



ML Based Spatial Denoising

Computed Motion Map(s)



Still Image vs Video->Video

- FastDVDNet changed several elements of this classical landscape.
- First, the warping stage (for the images surrounding the reference frame) was completely removed.
- Second, spatial denoising was replaced with “mini” temporal denoising, by feeding 3 consecutive temporal frames into an ML based denoising unit.
- These outputs were then fed into a second stage ML denoiser.
- Third, a modified version of the UNet architecture was used (identity skip connection between the I/O and skip concatenation replaced with skip summation).

Still Image vs Video->Video

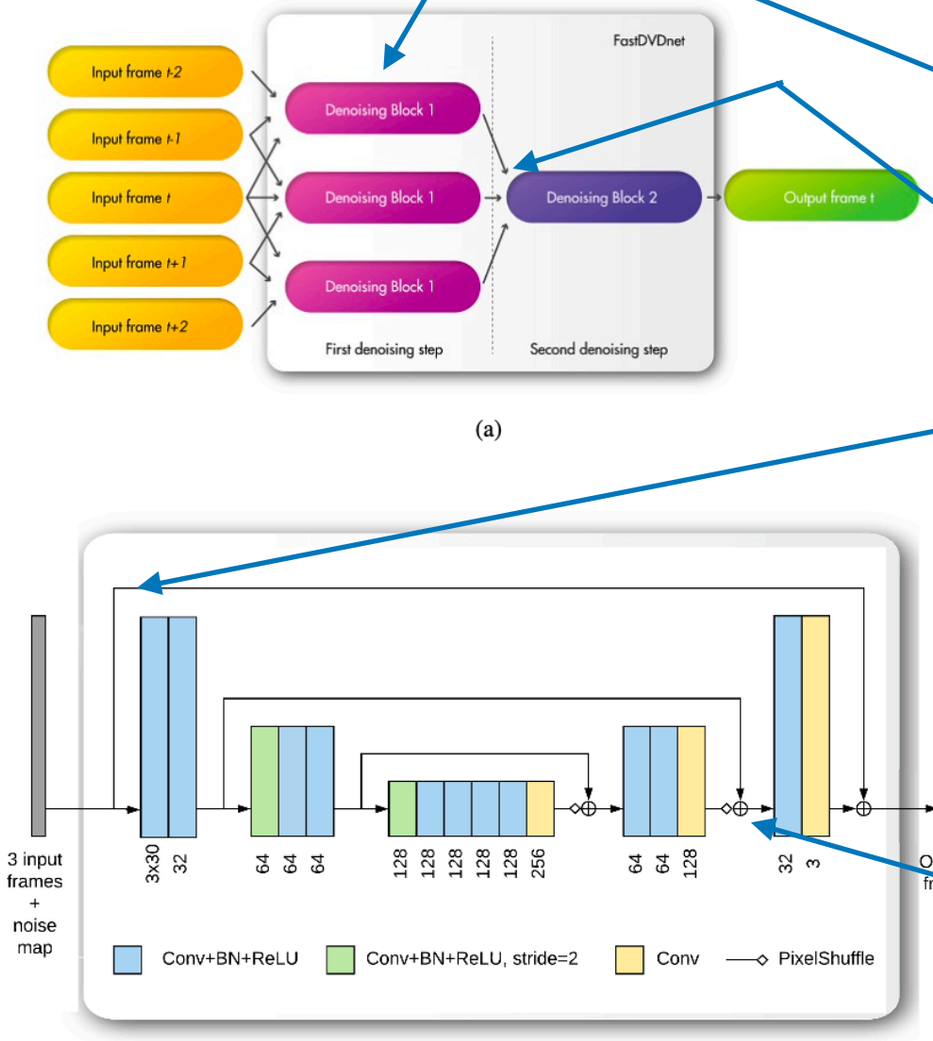


Figure 1. *Architecture used in FastDVDnet.* (a) A high-level diagram of the architecture. Five consecutive frames are used to denoise the middle frame. The frames are taken as triplets of consecutive frames and input to the *Denoising Blocks 1*. The instances of these blocks have all the same weights. The triplet composed by the outputs of these blocks are used as inputs for *Denoising Block 2*. The output of the latter is the estimate of the central input frame (*Input frame t*). Both *Denoising Block 1* and *Denoising Block 2* share the same architecture, which is shown in (b). The denoising blocks of FastDVDnet are composed of a modified multi-scale U-Net.

ML based denoiser now consumes 3 input frames vs 1.

Warping is no longer applied.

Identity skip connection is added.

Skip concatenation by summation is used.

Still Image vs Video->Video

Comments

- The most interesting contribution, was the removal of the warping stage.
- Unfortunately, this was not done in isolation with a corresponding ablation study.
- TODO: Perform a more detailed investigation to determine:

1) possible regressions on different video sequences

2) isolate the first order modifications, permitting the removal of the warping stage.

Still Image vs Video->Video

- Another way to remove the warping stage / motion map was introduced in VLNet.

- i.e. Self similarity was applied across each pixel location in space and time.

- A 1xN vector was then generated for each pixel location, using the center pixel value for each similar patch.

- The volumetric tensor was then fed into a CNN network, and then DnCNN, resulting in a residual noise map, for the reference frame.

Still Image vs Video->Video

Classical similarity search used in non local means BM3D.

New heuristic network

Previously discussed DnNN for image based residual noise map computation

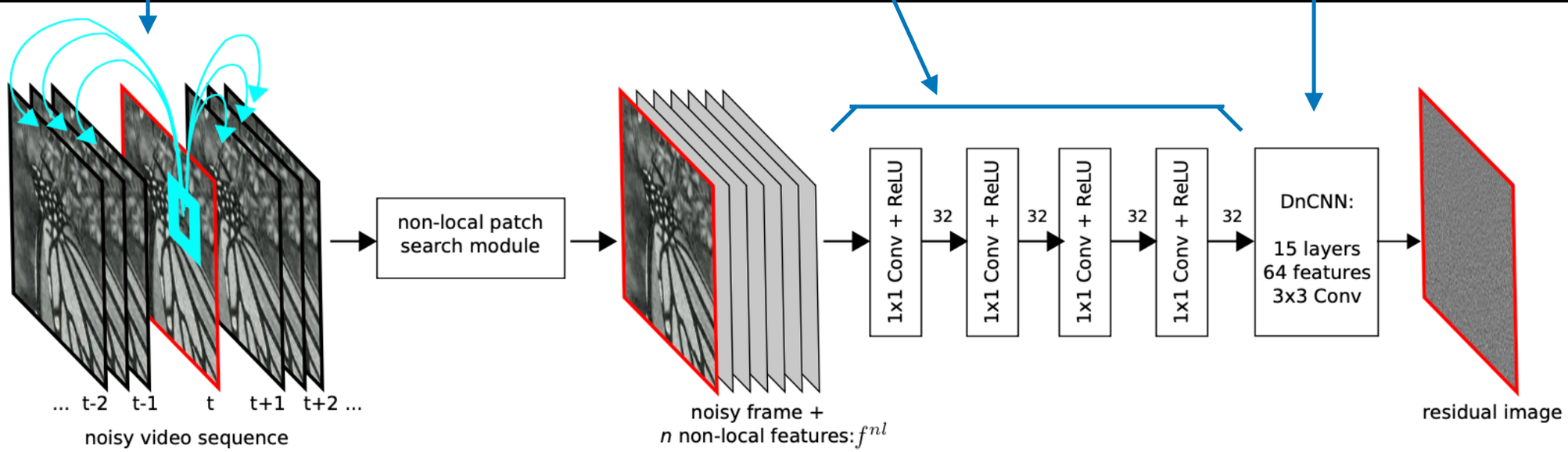


Figure 1: The architecture of the proposed method. The first module performs a patch-wise nearest neighbor search across neighboring frames. Then, the current frame, and the feature vectors f^{nl} of each pixel (the center pixels of the nearest neighbors) are fed into the network. The first four layers of the network perform 1×1 convolutions with 32 feature maps. The resulting feature maps are the input of a simplified DnCNN [55] network with 15 layers.

Image Generation by Reverse Diffusion

Use a generative Markov chain to convert one distribution to another (used in non-equilibrium statistical physics):

- “Systematically destroy structure in a target (data) distribution using an iterative forward diffusion process.”
- i.e. Perform (gaussian, binomial, …) diffusion into a (gaussian binomial, …) distribution.
- Then, build a generative Markov chain that converts the known distribution (gaussian, binomial, …) into the target (data) distribution via an iterative reverse diffusion process.
- i.e. “Learn a reverse diffusion process that restores structure in the data, yielding a highly flexible and tractable generative model of the data.”
- Do this by estimating the small perturbations in the reverse diffusion process.

Image Generation by Reverse Diffusion

- At every time step, we know the injected noise used in the forward diffusion process.

- Starting from a noisy image / a known noise distribution, basic structure is then hallucinated in the reverse process.

- As we iterate and noise is removed, we revert to a family of plausible results / images.

Image Generation by Reverse Diffusion

The details:

- Ho, et. al., fixes the forward diffusion process, by adding known gaussian noise, according to a fixed schedule.

- In the reverse process, the authors examine both reconstruction and residual denoising, by predicting both quantities: $u_t^{\sim} \; and \; \epsilon$

- The author cites that in order for the reconstruction approach to be success, training needs to be done using the variational bound.

- In contrast, for residual learning, training can be done using MSE loss.

- Comparable results were obtained using both approaches.

Image Generation by Reverse Diffusion vs Image Denoising by Anisotropic Diffusion

- Both diffusion processes denoise images.

- Both diffusion processes are rooted in evolutionary mathematical models.

Forward / reverse diffusion is based on non-equilibrium statistical physics and Markov chains.

Anisotropic diffusion is based on a partial differential equation - the non-linear heat equation.

- Both processes require incremental time steps, to accomplish their objective.

Image Generation by Reverse Diffusion vs Image Denoising by Anisotropic Diffusion

- i.e. T = 1000 time steps are used in forward / reverse diffusion.
- i.e. The largest update “time” step for anisotropic diffusion is 1/4.
- Using coarser sampling in forward / reverse diffusion or larger time steps in anisotropic diffusion will degrade results, leading to potential instability.
- Whereas forward / reverse diffusion is about injecting and removing gaussian noise, anisotropic diffusion is about removing noise through edge preserving filtering / adaptive smoothing.
- Whereas forward / reverse diffusion relies on a forward and a reverse process, anisotropic diffusion is only well posed in the forward spatial filtering direction.

Image Generation by Reverse Diffusion vs Image Denoising by Anisotropic Diffusion

- Reverse anisotropic diffusion is an ill posed process.
- In theory, reverse anisotropic diffusion corresponds to image sharpening.
- Food for thought: Can reverse anisotropic diffusion be stabilized and made well posed, by “tracking” the applied blur from the forward process, and learning an incremental sharpening for the inverse process.
- i.e. An incremental unsharp masking?

References

- Image Denoising by Sparse 3-D Transform Domain Collaborative Filtering, Dabov, et. al., IEEE Trans. Image Processing, 2007

- Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising, Zhang, et. al., IEEE Trans. Image Processing, 2017

- FFDNet: Toward a Fast and Flexible Solution for CNN Based Image Denoising, Zhang, et. al., IEEE Trans. Image Processing, 2017

- Towards Controllable Real Image Denoising with Camera Parameters, Oh, et. al., Int. Conference Image Processing, 2025

- UNet: Convolutional Networks for Biomedical Image Segmentation, Ronnenberger, et. al., International Conference on Medical Image Computing and Computer-Assisted Intervention, 2015

- The Transformation of Poisson, Binomial and Negative Binomial Data, F.J. Anscombe, Biometrika, 1948

References

- Noise2Noise: Learning Image Restoration without Clean Data, Lehtinen, et. al., ICML 2018

- Noise2Void: Learning Denoising from Single Noisy Images, Krull, et. al., CVPR 2019

- FastDVDNet: Towards Real-Time Deep Video Denoising Without Flow Estimation, Tassano, et. al., CVPR 2020

- (VNLNet) Non Local Video Denoising by CNN, Davy, et. al., arXiv, 2018 DVDNet: A Fast Network for Deep Video Denoising, Tassano, et. al., Int Conference Image Processing, 2019

- Denoising Diffusion Probabilistic Models, Ho, et. al., NIPS, 2020

- Deep Unsupervised Learning Using Non Equilibrium Thermodynamics, Sohl-Dickstein, ICML, 2015